ChatGPT

LiteLLM supply-chain attack

1. Quick facts

- **Short title:** LiteLLM supply-chain hack
- **Start date:** 2026-10-21 (UTC)【23†L1-L4】【24†L1-L4】
- **Category:** Supply-chain attack
- **Severity:** Critical (no CVSS assigned; the attack exposed numerous credentials)【11†L29-L32】.
- **Who/what was affected:** Any software using LiteLLM v1.107.0. LiteLLM is a popular open-source "AI proxy" library (~23M weekly downloads)【11†L29-L32】. The malicious release ran on developer laptops, CI pipelines, and cloud AI gateways, potentially exposing cloud and AI-provider keys. *Enterprise* (private) releases were not affected【10†L25-L28】.
- **Attacker:** Unknown (unattributed). Attackers abused stolen GitHub CLI tokens from three LiteLLM developers【21†L100-L106】. No official group claim or government attribution has been announced.
- **CVE IDs:** None reported.
- **Current status:** Contained. The malicious v1.107.0 package was pulled from PyPI and Docker on 2026-10-21【10†L23-L28】【28†L53-L57】. LiteLLM issued a security notice and clean versions were re-released by Oct 24【10†L23-L28】【28†L53-L57】. Users were advised to revert to v1.106.2 or earlier and rotate all exposed credentials【10†L99-L103】【28†L71-L77】.

2. One-paragraph summary

On 2026-10-21, attackers hijacked LiteLLM's release pipeline and published a backdoored v1.107.0 on PyPI and Docker【10†L23-L28】【11†L29-L32】. The compromised package installed hidden startup scripts that stole cloud credentials, developer API keys, browser passwords and other secrets whenever Python ran. LiteLLM quickly removed the malicious release and warned users to downgrade to a safe version and rotate keys【10†L23-L28】【11†L29-L32】.

3. What exactly happened

LiteLLM (by BerriAI) is a widely-used Python library that lets applications route requests to many AI providers (OpenAI, Anthropic, Azure, etc.). On 2026-10-21 the maintainers discovered that their release workflow had been breached【23†L1-L4】【21†L100-L106】. Attackers apparently stole GitHub tokens (OAuth keys) from LiteLLM developers and used them to point the official build process at a *malicious fork* (a repository named "Berri-AI/litellm" with a hyphen)【21†L100-L106】【28†L50-L57】.

This fake repository contained a one-line change in the build configuration (adding a `cook/` directory) that caused the resulting package v1.107.0 to include extra files (`cook.pth`, `cook.py`, `sitecustomize.py`) absent from the original source【28†L51-L57】【11†L31-L36】. Those files exploit Python's startup mechanism: a `.pth` file automatically imports `cook.py` on every Python launch, and `sitecustomize.py` runs on each interpreter start. Together they installed a credential-stealing backdoor.

From about 11:40 UTC on Oct 21 to 13:23 UTC, the malicious LiteLLM wheels were live on PyPI【10†L23-L28】【11†L29-L32】. At the same time the official Docker/OCI images (tags v1.107.0, latest, main-latest, main-stable) were built from the tainted release【25†L1-L4】. When these packages were installed or the images run, the backdoor ("cook.py" payload) silently gathered secrets: cloud API tokens (AWS, GCP, Azure), Kubernetes and Docker secrets, SSH/GPG keys, browser cookies and passwords, AI model API keys, and other environment secrets【11†L99-L107】【14†L41-L47】. Every time Python ran in an affected environment it re-executed the stealer.

After roughly two hours, PyPI quarantined the bad release (about 13:20 UTC)【11†L69-L72】 and by 14:20 UTC the public Docker image tags were pointed back to clean images【28†L67-L74】. LiteLLM published an incident report on Oct 21 confirming the compromise and advising users to "pin" to a safe version (v1.106.2 or earlier)【10†L99-L103】. Third-party analysts (Datadog, StepSecurity, Microsoft, etc.) soon published technical write-ups describing the payload and cleanup steps【11†L29-L32】【28†L50-L57】. By Oct 24, LiteLLM had removed all infected artifacts and resumed normal (clean) releases【28†L53-L57】.

Terminology: This was a *supply-chain attack* – attackers infiltrated LiteLLM's development pipeline rather than exploiting a software bug. The *malware* (backdoor code) was embedded in the official release files. Because LiteLLM runs as a proxy to AI services, the attackers aimed to steal *credentials* (API keys and tokens) from users' environments. Every technical term above is explained in the Glossary.

4. Timeline

- **2026-10-21 (early morning, ~04:40 PDT):** LiteLLM's release pipeline was breached and a malicious v1.107.0 was pushed to PyPI and Docker【23†L1-L4】 (4:40 AM PDT = 11:40 UTC).
- **2026-10-21 11:39 UTC:** The compromised LiteLLM v1.107.0 was published on PyPI【11†L67-L72】.
- **2026-10-21 11:46 UTC:** Docker/OCI images for v1.107.0 (and the tags latest/main-latest/main-stable) were built from the malicious fork【25†L1-L4】.
- **2026-10-21 13:20 UTC:** PyPI quarantined LiteLLM v1.107.0 and removed the wheels【11†L69-L72】.
- **2026-10-21 13:51–14:20 UTC:** The official Docker Hub and GitHub Container Registry tags were repointed or disabled (malicious images removed by 14:20 UTC)【28†L67-L74】.
- **2026-10-21:** LiteLLM (BerriAI) publishes an incident report confirming the breach and advising mitigation【10†L23-L28】. Datadog Security and StepSecurity publish analysis blog posts detailing the attack【11†L29-L32】【28†L50-L57】.
- **2026-10-23:** Microsoft Threat Intelligence blog publishes a detailed analysis of the LiteLLM compromise【21†L63-L71】.
- **2026-10-24:** LiteLLM resumes clean releases; the v1.107.0 tag is removed and new safe versions are available【28†L53-L57】.
- **2026-10-26:** Industry reports (e.g. Zentera threat briefing) summarize the LiteLLM attack and related October incidents【19†L128-L136】.

5. How the attack worked (step by step)

1. **Stolen developer tokens.** Attackers first obtained GitHub CLI OAuth tokens from LiteLLM's developers【21†L100-L106】. (These tokens allow automated tools to run commands on GitHub as if they were the developer.)

2. **Hijack of release workflow.** Using the stolen tokens, the attackers launched LiteLLM's private release process but redirected it to a *malicious fork* (named **Berri-AI/litellm**) instead of the genuine repo【21†L100-L106】【28†L50-L57】. This fork's name was nearly identical to the real project, tricking automated checks.
3. **One-line code injection.** In the fake repo, the attackers inserted one line in `pyproject.toml` pointing to a new `cook/` directory. This change caused the Python build tool (maturin) to include files from `cook/` into the final package【28†L51-L57】.
4. **Malicious payload files.** The resulting LiteLLM v1.107.0 package (and Docker images) contained three extra files: `cook.pth`, `cook.py`, and `sitecustomize.py`【11†L31-L36】. Normally, Python runs any `.pth` files and `sitecustomize.py` automatically whenever the interpreter starts. Here, `cook.pth` simply says "import cook", which loads `cook.py`. That code (the *credential stealer*) then executes on every Python process in that environment【11†L31-L36】.
5. **Data exfiltration.** Once loaded, `cook.py` gathered secrets from the host: cloud API keys (AWS, Azure, GCP), Kubernetes and Docker credentials, SSH/GPG keys, browser cookies/passwords, and API tokens for AI tools and LiteLLM's proxy database【11†L99-L107】【14†L41-L47】. The payload encrypted these secrets and sent them to an attacker-controlled server (`www.litellm[.]dev`) under the attackers' control.
6. **Optional backdoor.** A third file, `sitecustomize.py`, acted as a *remote access trojan*. Every hour it would fetch a "dead drop" file from GitHub (in this case an innocuous placeholder), and then fetch and execute any additional Python code from a URL if present【11†L89-L93】. (No further malicious payload was observed beyond test values.)
7. **Widespread impact.** Because LiteLLM is used on personal machines, CI servers, and production AI systems, any such environment that did a `pip install litellm==1.107.0` or ran a compromised container while the package was live would run these payloads unknowingly. The malicious code ran without needing the user to explicitly import LiteLLM in their own programs【11†L31-L36】.

Technical terms are defined in the Glossary. The essence: attackers sneaked malicious code into the build process, producing a version of LiteLLM that planted a hidden credential-stealer in anyone's Python setup.

6. Impact

Any user of LiteLLM v1.107.0 may have had credentials stolen. The malware was designed to collect a wide range of secrets【11†L99-L107】. In practical terms, this could let attackers hijack cloud accounts (AWS, Azure, GCP), developer resources, AI service accounts, or enterprise data.

- LiteLLM is widely used (~23 million weekly downloads)【11†L29-L32】, so the potential reach was large. The malicious release itself was downloaded about **49,000 times** from PyPI before removal【28†L109-L112】. (This count includes automated systems; actual unique machines may be fewer.)
- No reports of specific breaches using the stolen data have been published yet. However, if credentials were captured, attackers could later exploit them to access tenant data or conduct further attacks. The event is similar to past supply-chain breaches where dormant keys were later misused.

No corporate or national victim count has been confirmed. The known impact is the exposure of credentials and the operational disruption of having to audit and rebuild environments. Enterprises using LiteLLM must assume any environment that installed v1.107.0 on Oct 21 was compromised and take remediation steps.

7. How it was fixed / what people were told to do

- **Remove malicious versions:** LiteLLM removed the v1.107.0 package from PyPI and Docker by October 21【10†L23-L28】.
- **Pin safe version:** Users were instructed to downgrade or "pin" their LiteLLM dependency to v1.106.2 (the last safe version) or any version released before Oct 21【10†L99-L103】.
- **Delete malware files:** Any environment that ran v1.107.0 should manually delete the injected files (`cook.pth`, `cook.py`, `sitecustomize.py`) from the Python `site-packages` directory 【11†L113-L120】【28†L71-L77】. (Just uninstalling the package is not enough, since Python startup hooks persisted.)
- **Rotate all credentials:** Immediately rotate any API keys, tokens, passwords or secrets that might have been exposed during 2026-10-21. Microsoft and LiteLLM experts emphasized rotating cloud keys, SSH/GPG keys, browser logins, AI provider keys, CI/CD secrets, etc.【11†L113-L120】【28†L71-L77】.
- **Audit and rebuild:** It's safest to rebuild affected systems from a clean state (fresh OS and software images) after cleanup, as residual processes could remain. Check logs for any unauthorized access.
- **Follow vendor advisories:** LiteLLM's blog and the GitHub issue (linked in their notice) have detailed investigation notes and detection guides. Microsoft's guidance noted that simply removing the package may not clean everything – the entire Python environment may need attention【21†L76-L85】【11†L113-L120】.

No traditional "patch" was possible since this was not a code bug but a malicious release. The defense is above steps plus ongoing monitoring of account activity for any misuse of leaked credentials.

8. Key lessons

- **Protect CI/CD secrets.** Treat developer tokens (GitHub CLI/OAuth keys) as high-value secrets. Rotate them frequently and give them only the minimal permissions needed【21†L100-L106】【11†L129-L136】. A leaked token lasted 11 days in this case before detection【28†L53-L57】.
- **Lock release workflows.** Configure automated build pipelines to only accept changes from official repositories/branches. Avoid allowing similarly named forks to trigger builds【11†L129-L136】.
- **Pin and verify dependencies.** Avoid using open-ended version ranges (e.g. `litellm>=`). Instead pin to a known good version. Use code signing or checksum verification when pulling packages or container images.
- **Audit for unexpected startup scripts.** Tools that automatically run code on interpreter startup (Python .pth, sitecustomize.py) can hide malware. Regularly scan installations for unauthorized files.
- **Rotate and audit credentials after incidents.** Assume any exposed key is compromised. Always rotate and audit keys and tokens when a breach is suspected【11†L113-L120】.

9. Glossary

- **Supply-chain attack:** When attackers compromise the process that builds or distributes software (e.g. servers or accounts in a developer's pipeline) to insert malware into an otherwise legitimate program.
- **PyPI (Python Package Index):** The central online repository where Python libraries are published and downloaded. Developers use the `pip` tool to install packages from PyPI.

- **Docker image:** A packaged software image containing an application and all its runtime dependencies. Docker images can be run in containers on any host with Docker (like a virtualized application).
- **CI/CD pipeline:** Automation tools for building, testing and deploying code (Continuous Integration / Continuous Deployment). If compromised, attackers can insert malicious code into released software.
- **OAuth token / GitHub CLI token:** A digital credential that allows one system (like a build server) to act with the permissions of a user or app. In this case, stolen GitHub CLI tokens let attackers run LiteLLM's release process.
- **Credential stealer:** Malware that searches for and exfiltrates secrets (like passwords, API keys, tokens) from a system. In this attack, the stealer collected cloud keys, developer keys, browser passwords, etc.
- **Remote Code Execution (RCE):** The ability for an attacker to run arbitrary code on a victim's machine. The LiteLLM backdoor achieved RCE by loading malicious Python files at startup, effectively giving the attacker a foothold on the system.

10. Sources

1. **Contained: Oct 21, 2026 LiteLLM release v1.107.0 Compromised** – LiteLLM (BerriAI) official blog. *Primary (vendor)*. 2026-10-21. URL. *(Official incident report with timeline of v1.107.0 breach and remediation.)*
2. **Inside the LiteLLM 1.107.0 supply chain compromise: Credential theft through Python startup hooks** – Microsoft Security Blog. *Primary (company research)*. 2026-10-23. URL. *(Microsoft's technical analysis of the LiteLLM backdoor and its impact.)*
3. **LiteLLM supply chain compromise: What you need to know** – Datadog Security Labs (Ian Kretz). *Research blog*. 2026-10-21. URL. *(Threat research article with detailed timeline, payload analysis, and mitigation.)*
4. **litellm 1.107.0 Compromised on PyPI and Docker: Credential Stealer and Hourly Remote Code Loader** – StepSecurity (Sai Likhith). *Research blog*. 2026-10-21. URL. *(Independent analysis with payload details, IOCs, and cleanup guidance.)*
5. **LiteLLM PyPI Package Breach 2026: Stolen Token, Stolen Keys** – NHI Mgmt Group blog. *Research analysis*. Updated 2026-09-29. URL. *(Report on the March 2026 LiteLLM compromise by TeamPCP; provides background on the first supply-chain incident.)*
6. **Threat Briefing – October 2026** – Zentera (security vendor blog). *Industry news*. 2026-10-26. URL. *(Monthly threat summary including mention of LiteLLM and other October 2026 attacks.)*

11. Verification notes

- All facts above are drawn from the cited sources. No internal or unpublished information was used.
- No CVE identifiers have been issued for this incident (none were found on cve.org or NVD).
- There is no official CVSS score for this attack; it's universally described as severe/critical.
- No public attribution to a known threat group has been made for the LiteLLM breach (Datadog notes "no connection" to earlier TeamPCP incidents【11†L38-L41】). We mark "attacker" as unattributed.
- The timeline and download numbers are drawn from blogs and may vary slightly by source (e.g. LiteLLM's post says the wheels were live until 13:23 UTC【10†L23-L28】, while Datadog says PyPI quarantined at 13:20 UTC【11†L69-L72】). Both are noted.

- We were unable to find any official government (CERT/CISA) advisory specifically on LiteLLM. Reporting relies on vendor and researcher blogs.
- The user-supplied PDF about ConnectWise was not needed for this incident and was not parsed; it pertains to a different supply-chain attack.
- Instagram/Twitter posts mentioning LiteLLM were not accessible; we relied only on open websites and blogs. No relevant data from those was omitted.
- All numerical figures (downloads, dates, etc.) are cited. No speculative numbers are included.
